

Reviewer Pack — Tropical Positive-Scalar Homogeneity of MinimalFourierCoeffi...

Entry 6be01f5e483c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 02:28:30 UTC

Tropical Positive-Scalar Homogeneity of MinimalFourierCoefficient

Entry ID: 6be01f5e483c **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 02:28:30 UTC

1. Conjecture

Field A (mathematical branch): TROPICAL_FOURIER_ANALYSIS (tropical geometry $\times$ Fourier analysis over the max-plus semiring) **Field B** (complexity object): Min-plus circuit complexity / tropical circuit lower bounds (a complexity-theoretic measure governing shortest-path and dynamic-programming circuits)

Statement:

Let f be a TropicalPolynomial on a finite grid $X \subset \mathbb{R}^d$, and let $\lambda > 0$. Define the tropically rescaled polynomial $g = \lambda \odot f$, i.e., the pointwise rescaling $g(x) = \lambda \cdot f(x)$ (tropical exponentiation by λ). Let $F = \text{TropicalFourierTransform}(f)$ and $G = \text{TropicalFourierTransform}(g)$ be their tropical Fourier coefficients (the Legendre–Fenchel-type transforms guaranteed invertible by axiom A2 on the relevant class). Then $\text{MinimalFourierCoefficient}(g) = \lambda \cdot \text{MinimalFourierCoefficient}(f)$, and $\text{DiscrepancyCalculation}(g) = \lambda \cdot \text{DiscrepancyCalculation}(f)$. In particular, the bound from axiom A3, $\text{DiscrepancyMeasure}(h) \leq \max_k |\hat{F}_h(k)|$, scales homogeneously of degree 1 under positive tropical rescaling, so the inequality is preserved exactly under $\lambda > 0$ (no slack opens or closes).

Rationale (proposer’s reasoning):

This conjecture directly tests axiom A3 ($\text{DiscrepancyMeasure} \leq \max \text{Fourier coefficient}$) jointly with axiom A2 (invertibility of $\text{TropicalFourierTransform}$ on the chosen class). Because tropical multiplication by a positive scalar corresponds to ordinary multiplication, and the Legendre–Fenchel transform satisfies $(\lambda \cdot f)(y) = \lambda \cdot f(y/\lambda)$, both extremes of f^* (its minimum and the discrepancy it controls) must scale by exactly λ . Verifying this 1-homogeneity is a clean sanity check that the framework’s notion of $\text{MinimalFourierCoefficient}$ behaves as a genuine ‘norm-like’ invariant compatible with the tropical semiring’s scalar action, and that the A3 bound is not an artifact of a particular scale (relevant for transferring lower bounds in tropical/min-plus circuit complexity, where rescaling weights is a standard reduction).

Taxonomy category: FOURIER_ANALYTIC (status at proposal time:)

Framework membership: framework fw_28b4bfb95f, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d6a1dbab22779fb0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 500 random tropical polynomials $\times 10$ λ samples (5000 trials, seed-controlled), conjecture is SUPPORTED iff every trial satisfies $|MFC(\lambda \cdot f) - \lambda \cdot MFC(f)| < 1e-9$ AND $|Disc(\lambda \cdot f) - \lambda \cdot Disc(f)| < 1e-9$ AND $|slack(\lambda \cdot f) - \lambda \cdot slack(f)| < 1e-9$, where $slack = \max_y |F_h(y)| - Disc(h) \geq 0$ for all trials.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.97	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical Fourier transform Legendre-Fenchel min-plus polynomial - tropical circuit complexity min-plus homogeneity scaling lower bound - max-plus semiring Fourier coefficient discrepancy tropical polynomial

Top relevant hits considered: - [http://arxiv.org/abs/1010.5964v1] Quadratic discrete Fourier transform and mutually unbiased bases - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1204.4578v1] Complexity of tropical and min-plus linear prevarieties - [http://arxiv.org/abs/1406.3065v2] Lower Bounds for Tropical Circuits and Dynamic Programs - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/1606.05050v1] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [http://arxiv.org/abs/1503.01392v2] Valuations of Semirings - [http://arxiv.org/abs/0812.3496v1] Linear independence over tropical semirings and beyond

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import json
from itertools import product

def add(a, b):
    return [x + y for x, y in zip(a, b)]

def sub(a, b):
```

```

    return [x - y for x, y in zip(a, b)]

def max_elem(lst):
    return max(lst)

def mean(lst):
    return sum(lst) / len(lst)

def tropical_add(f, g):
    return [max(a, b) for a, b in zip(f, g)]

def tropical_mul(f, c):
    return [c * x for x in f]

def tropical_fourier_transform(f, grid_points):
    d = len(grid_points)
    grid_size = len(grid_points[0])
    F = [0] * (grid_size ** d)

    def index(coords):
        return sum((coords[i] - grid_points[i][0]) * grid_size**i for i in range(d))

    for x in product(*grid_points):
        y = sub(x, [1, 1, 1]) # Adjusted to avoid negative indices
        F[index(y)] = max(F[index(y)], sum(tropical_mul(monomial, x) for monomial in f))

    return F

def discrepancy_calculation(f, grid_points):
    d = len(grid_points)
    grid_size = len(grid_points[0])
    total = 0
    count = 0

    def index(coords):
        return sum((coords[i] - grid_points[i][0]) * grid_size**i for i in range(d))

    for x in product(*grid_points):
        total += f[index(x)]
        count += 1

    mean_f = total / count
    return max_elem(f) - mean_f

def run_trial(seed: int) -> dict:
    random.seed(seed)
    d = random.randint(1, 2)
    grid_size = 41
    grid_points = [[-20 + i * (grid_size // 40) for i in range(grid_size)] for _ in range(d)]

    f = [max([random.randint(-10, 10) * x[i] + random.randint(-10, 10) for i in range(d)]) for _ in range(grid_size)]

    MFC_f = min(tropical_fourier_transform(f, grid_points))
    Disc_f = discrepancy_calculation(f, grid_points)

    lambda_values = [random.uniform(0.1, 5) for _ in range(10)]

```

```

results = []

for  $\lambda$  in lambda_values:
    g = tropical_mul(f,  $\lambda$ )
    MFC_g = min(tropical_fourier_transform(g, grid_points))
    Disc_g = discrepancy_calculation(g, grid_points)

    results.append({
        "lambda":  $\lambda$ ,
        "MFC_f": MFC_f,
        "MFC_g": MFC_g,
        "Disc_f": Disc_f,
        "Disc_g": Disc_g
    })

all_correct = True
for result in results:
    if not (abs(result["MFC_g"] -  $\lambda$  * result["MFC_f"]) < 1e-9 and abs(result["Disc_g"] -  $\lambda$  * result["Disc_f"]) < 1e-9):
        all_correct = False

return {
    "metric_name": "Tropical Positive-Scalar Homogeneity",
    "metric_value": MFC_f,
    "instances_tested": len(results),
    "conjecture_holds": all_correct,
    "counterexample": "" if all_correct else "lambda-dependent discrepancy"
}

if __name__ == "__main__":
    seeds = [11, 23, 37, 53, 71] if not sys.argv[1:] else [int(s) for s in sys.argv[1:]]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='lambda-dependent discrepancy' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21db269d.py", line 110, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21db269d.py", line 71, in run_trial
    f = [max([random.randint(-10, 10) * x[i] + random.randint(-10, 10) for i in range(d)]) for _ in range(5000)]
           ^
NameError: name 'x' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a NameError ('x' is not defined') before any trials executed, so no data was produced to evaluate the pre-registered support or falsification conditions. | next: Fix the test harness by properly defining the grid points x within the polynomial generation (e.g., evaluate the affine forms over each grid point), then rerun the 5000 trials.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	8486
2	novelty	claude_max	opus	0	0	3670
3	novelty	claude_max	opus	0	0	9209
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15625
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12389
6	judge	claude_max	opus	0	0	6151

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 55531 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/6be01f5e483c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6be01f5e483c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/6be01f5e483c.tar.gz (if generated)